

SUPPORT DE COURS OFFICIEL & MASTERCLASS

DÉVELOPPEUR FULL STACK

MERN Stack, TypeScript, API REST, GraphQL & DevOps

Un guide ultra-complet, théorique, pratique et illustré pour maîtriser la conception, le développement, le test et le déploiement d'applications Web modernes, robustes et scalables.

Niveau : Débutant à Avancé / Master Class

Stack Technique : React, Node.js, Express, MongoDB, TypeScript, PostgreSQL, Prisma, Docker

Format : Théorie approfondie + Exercices corrigés + 3 Projets Fil Rouge

Édition : 2026 / Version Haute Qualité Professionnelle

Table des Matières Synthétique

Module 0 : Fondations du Web & Architecture Client-Serveur

- 0.1 Protocole HTTP/HTTPS, Verbes & Statuts
- 0.2 Le DNS, IP et le Cycle de Vie d'une Requête Web
- 0.3 Configuration de l'Environnement (VS Code, Git, Terminal)

Module 1 : Core Web Technologies (HTML5, CSS3, JS ES6+)

- 1.1 HTML Sémantique, Accessibilité (WCAG / A11y)
- 1.2 CSS Modernes : Flexbox, Grid & Responsive Layouts
- 1.3 JavaScript Asynchrone : Promises, Event Loop & Async/Await

Module 2 : Frontend Masterclass avec React & TypeScript

- 2.1 Architecture React, JSX & Virtual DOM
- 2.2 React Hooks en Profondeur (useState, useEffect, useMemo, useCallback, useRef)
- 2.3 TypeScript Intégré à React : Typage des Props, Events & State
- 2.4 Gestion d'État Global : Redux Toolkit vs Zustand

Module 3 : Backend Architecture avec Node.js & Express.js

- 3.1 Node.js Internals : Event Loop, Thread Pool & Non-Blocking I/O
- 3.2 Construction d'une API RESTful Industrielle en TypeScript
- 3.3 Middlewares, Validation Zod & Sécurité (JWT, Hachage Argon2, Helmet, Rate Limiting)

Module 4 : Persistence des Données (NoSQL & SQL)

- 4.1 MongoDB & Mongoose : Schemas, Aggregations & Indexes
- 4.2 PostgreSQL & Prisma ORM : Relations, Migrations & Transactions

Module 5 : DevOps, Containerisation & CI/CD

- 5.1 Containerisation avec Docker & Docker-Compose
- 5.2 Pipelines CI/CD avec GitHub Actions & Déploiement Cloud

Module 6 : Projets Pratiques Complètement Détaillés

-  Projet 1 : TaskManager Pro (REST API + React + Auth JWT)
-  Projet 2 : E-Commerce Platform Lite (Prisma + Stripe + Upload Cloud)
-  Projet 3 : Real-Time Chat Engine (WebSockets / Socket.io)

MODULE 0 : Fondations du Web & Architecture Client-Serveur

0.1 Le Modèle Client-Serveur & Protocole HTTP

Le développement web Full Stack repose fondamentalement sur le modèle **Client-Serveur**. Dans ce paradigme, le *Client* (généralement un navigateur web comme Chrome, Firefox ou Safari, ou une application mobile) envoie des requêtes réseau au *Serveur*, qui traite la demande, exécute la logique métier nécessaire, interroge une base de données au besoin, et retourne une réponse appropriée.

Anatomie d'une Requête HTTP

Une requête HTTP comprend :

- **Une Méthode/Verbe** : Indique l'action à réaliser (`GET` , `POST` , `PUT` , `PATCH` , `DELETE`).
- **Un URL / URI** : La ressource visée (ex: `https://api.app.com/v1/users`).
- **Des En-têtes (Headers)** : Métadonnées (ex: `Content-Type: application/json` , `Authorization: Bearer <token>`).
- **Un Corps (Body)** : Les données transmises (utilisé avec POST, PUT, PATCH).

Tableau Comparatif des Verbes HTTP

Verbe	Rôle Principal	Idempotent	Contient un Body ?
<code>GET</code>	Récupérer une ou plusieurs ressources sans modifier l'état serveur.	Oui	Non
<code>POST</code>	Créer une nouvelle ressource ou soumettre des données.	Non	Oui
<code>PUT</code>	Remplacer intégralement une ressource existante.	Oui	Oui
<code>PATCH</code>	Mettre à jour partiellement une ressource existante.	Non	Oui
<code>DELETE</code>	Supprimer une ressource spécifiée.	Oui	Optionnel

Codes de Statut HTTP Standardisés

- **2xx (Succès)** : `200 OK` (Requête réussie), `201 Created` (Ressource créée avec succès), `204 No Content` (Succès sans corps de réponse).
- **3xx (Redirection)** : `301 Moved Permanently` , `304 Not Modified` (Utilisé pour la mise en cache).
- **4xx (Erreurs Client)** : `400 Bad Request` (Syntaxe ou données invalides), `401 Unauthorized` (Non authentifié), `403 Forbidden` (Authentifié mais non autorisé), `404 Not Found` (Ressource introuvable).
- **5xx (Erreurs Serveur)** : `500 Internal Server Error` (Crash ou erreur non gérée sur le serveur), `503 Service Unavailable` (Serveur surchargé ou en maintenance).

0.2 DNS, Adressage IP et Cycle de Vie d'une Requête

Lorsque l'utilisateur saisit `https://www.example.com` dans son navigateur :

1. **Résolution DNS** : Le navigateur vérifie son cache, puis interroge le résolveur DNS de l'AFI pour traduire le nom de domaine en adresse IP (ex: `93.184.216.34`).
2. **Handshake TCP/IP & TLS** : Établissement d'une connexion sécurisée via le triple handshake TCP (SYN, SYN-ACK, ACK) et la négociation des clés de chiffrement TLS (HTTPS).
3. **Envoi de la Requête HTTP** : Le navigateur envoie la commande `GET / HTTP/1.1`.
4. **Traitement Serveur** : Le serveur web (Nginx, Apache, Node.js) reçoit la requête, consulte la base de données et prépare le document HTML / le JSON.
5. **Rendu Navigateur** : Le navigateur reçoit le flux de données, construit le DOM (Document Object Model) et le CSSOM, puis exécute le code JavaScript.

MODULE 1 : Frontend Standard (HTML5, CSS3, JavaScript ES6+)

1.1 HTML5 Sémantique & Accessibilité (A11y)

L'utilisation appropriée des balises sémantiques HTML5 garantit une meilleure référencement (SEO) ainsi qu'une accessibilité universelle aux lecteurs d'écran pour personnes malvoyantes.

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Plateforme e-Learning Full Stack</title>
</head>
<body>
  <header>
    <nav aria-label="Navigation Principale">
      <ul>
        <li><a href="/courses">Cours</a></li>
        <li><a href="/dashboard">Tableau de bord</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <article>
      <header>
        <h1>Maîtriser TypeScript en 2026</h1>
        <p>Publié le <time datetime="2026-09-25">25 Septembre 2026</time></p>
      </header>
      <section>
        <p>Le typage statique permet d'éviter plus de 15% des bugs en production...</p>
      </section>
    </article>
  </main>
</body>
</html>
```

1.2 Layouts CSS Modernes : Flexbox & Grid

Flexbox est idéal pour les agencements unidimensionnels (lignes ou colonnes), tandis que CSS Grid excelle dans la création de structures bidimensionnelles complexes.

```
/* Configuration CSS Grid pour une application Web responsive */
.dashboard-grid {
  display: grid;
  grid-template-columns: 250px 1fr;
  grid-template-rows: 60px 1fr 40px;
  grid-template-areas:
    "header header"
    "sidebar content"
    "footer footer";
  min-height: 100vh;
}

@media (max-width: 768px) {
  .dashboard-grid {
    grid-template-columns: 1fr;
    grid-template-areas:
      "header"
      "content"
      "footer";
  }
}
```

1.3 JavaScript Asynchrone : Event Loop & Async/Await

JavaScript est un langage mono-thread fonctionnant grâce à l'**Event Loop**. Les opérations coûteuses (I/O réseau, lecture de fichier) sont déléguées au système d'exploitation ou aux APIs du navigateur.

```
// Exemple d'interrogation d'API robuste avec Async/Await et gestion d'erreurs
interface UserProfile {
  id: string;
  name: string;
  email: string;
}

async function fetchUserProfile(userId: string): Promise<UserProfile> {
  try {
    const response = await fetch(`https://api.example.com/users/${userId}`, {
      method: 'GET',
      headers: {
        'Accept': 'application/json',
        'Authorization': 'Bearer token_secret'
      }
    });

    if (!response.ok) {
      throw new Error(`Erreur HTTP Status: ${response.status}`);
    }

    const data: UserProfile = await response.json();
    return data;
  } catch (error) {
    console.error("Échec de la récupération du profil:", error);
    throw error;
  }
}
```

MODULE 2 : Frontend Masterclass avec React & TypeScript

2.1 Concepts Clés : JSX, Virtual DOM & React Fiber

React utilise un **Virtual DOM**. Lors d'un changement d'état (State Update), React crée un nouvel arbre virtuel, calcule la différence avec le précédent via l'algorithme de **Reconciliation** (Diffing algorithm), puis effectue uniquement les mises à jour minimales nécessaires sur le DOM réel.

2.2 Les React Hooks Fondamentaux & Sur-Mesure

Les Hooks permettent d'utiliser l'état et d'autres fonctionnalités de React sans écrire de classes.

Composant Complexe avec TypeScript & Hooks

```

import React, { useState, useEffect, useMemo, useCallback } from 'react';

interface Product {
  id: number;
  title: string;
  price: number;
  category: string;
}

export const ProductCatalog: React.FC = () => {
  const [products, setProducts] = useState<Product[]>([]);
  const [search, setSearch] = useState<string>('');
  const [loading, setLoading] = useState<boolean>(true);

  useEffect(() => {
    let isMounted = true;
    fetch('https://fakestoreapi.com/products')
      .then(res => res.json())
      .then((data: Product[]) => {
        if (isMounted) {
          setProducts(data);
          setLoading(false);
        }
      });
    return () => { isMounted = false; };
  }, []);

  // Optimisation de calcul avec useMemo
  const filteredProducts = useMemo(() => {
    return products.filter(p =>
      p.title.toLowerCase().includes(search.toLowerCase())
    );
  }, [products, search]);

  // Callback mémorisé
  const handleSearchChange = useCallback((e: React.ChangeEvent<HTMLInputElement>) => {
    setSearch(e.target.value);
  }, []);

  if (loading) return <div>Chargement des produits en cours...</div>;

  return (
    <div style={{ padding: '20px' }}>
      <h2>Catalogue de Produits ({filteredProducts.length})</h2>
      <input
        type="text"
        placeholder="Rechercher un produit..."
        value={search}
        onChange={handleSearchChange}
        style={{ padding: '8px', width: '100%', marginBottom: '15px' }}
      />
      <ul>
        {filteredProducts.map(p => (
          <li key={p.id}>
            <strong>{p.title}</strong> - {p.price} €
          </li>
        ))}
      </ul>
    </div>
  );
};

```

```
);  
};
```

2.3 Gestion d'État Global Moderne avec Zustand

Zustand offre une alternative légère et intuitive à Redux Toolkit, exempte de boilerplate excessif.

```
import { create } from 'zustand';  
  
interface CartItem {  
  id: number;  
  name: string;  
  quantity: number;  
  price: number;  
}  
  
interface CartState {  
  items: CartItem[];  
  addToCart: (item: Omit<CartItem, 'quantity'>) => void;  
  removeItem: (id: number) => void;  
  clearCart: () => void;  
}  
  
export const useCartStore = create<CartState>((set) => ({  
  items: [],  
  addToCart: (product) => set((state) => {  
    const existing = state.items.find(i => i.id === product.id);  
    if (existing) {  
      return {  
        items: state.items.map(i =>  
          i.id === product.id ? { ...i, quantity: i.quantity + 1 } : i  
        )  
      };  
    }  
    return { items: [...state.items, { ...product, quantity: 1 }] };  
  })),  
  removeItem: (id) => set((state) => ({  
    items: state.items.filter(i => i.id !== id)  
  })),  
  clearCart: () => set({ items: [] })  
}));
```

MODULE 3 : Backend Architecture avec Node.js & Express

3.1 Architectures des APIs RESTful & TypeScript

Une architecture propre découple le routage, la logique métier (Services) et l'accès aux données (Repositories/Controlllers).

```
// Architecture en Couches (Controller - Service - Repository)
// src/controllers/user.controller.ts

import { Request, Response, NextFunction } from 'express';
import { UserService } from '../services/user.service';
import { z } from 'zod';

const createUserSchema = z.object({
  email: z.string().email("Format email invalide"),
  password: z.string().min(8, "Le mot de passe doit faire au moins 8 caractères"),
  name: z.string().min(2)
});

export class UserController {
  private userService = new UserService();

  public createUser = async (req: Request, res: Response, next: NextFunction): Promise<void>
=> {
    try {
      const validatedData = createUserSchema.parse(req.body);
      const newUser = await this.userService.registerUser(validatedData);
      res.status(201).json({
        success: true,
        data: newUser
      });
    } catch (error) {
      next(error); // Transmis au Middleware de gestion d'erreur globale
    }
  };
}
```

3.2 Authentification & Sécurité Industrielle (JWT + Argon2)

Pour sécuriser les mots de passe, **Argon2** (gagnant du Password Hashing Competition) est la recommandation actuelle, surclassant Bcrypt.

```

import argon2 from 'argon2';
import jwt from 'jsonwebtoken';

// Enregistrement d'un utilisateur
export async function hashPassword(plainPassword: string): Promise<string> {
  return await argon2.hash(plainPassword, {
    type: argon2.argon2id,
    memoryCost: 2 ** 16, // 64 MB
    timeCost: 3
  });
}

// Vérification
export async function verifyPassword(hash: string, plain: string): Promise<boolean> {
  return await argon2.verify(hash, plain);
}

// Génération de Token JWT Sécurisé
export function generateTokens(userId: string, role: string) {
  const accessToken = jwt.sign(
    { sub: userId, role },
    process.env.JWT_SECRET || 'secret_key_default',
    { expiresIn: '15m' }
  );

  const refreshToken = jwt.sign(
    { sub: userId },
    process.env.JWT_REFRESH_SECRET || 'refresh_secret_default',
    { expiresIn: '7d' }
  );

  return { accessToken, refreshToken };
}

```

MODULE 4 : Persistence des Données (NoSQL & SQL)

4.1 Modélisation NoSQL avec MongoDB & Mongoose

MongoDB est une base orientée document JSON/BSON, offrant une flexibilité horizontale idéale pour les données non structurées ou semi-structurées.

```

import { Schema, model, Document } from 'mongoose';

export interface IOrder extends Document {
  customer: Schema.Types.ObjectId;
  products: Array<{
    product: Schema.Types.ObjectId;
    quantity: number;
    priceAtPurchase: number;
  }>;
  totalAmount: number;
  status: 'pending' | 'paid' | 'shipped' | 'cancelled';
  createdAt: Date;
}

const OrderSchema = new Schema<IOrder>({
  customer: { type: Schema.Types.ObjectId, ref: 'User', required: true, index: true },
  products: [{
    product: { type: Schema.Types.ObjectId, ref: 'Product', required: true },
    quantity: { type: Number, required: true, min: 1 },
    priceAtPurchase: { type: Number, required: true }
  }],
  totalAmount: { type: Number, required: true },
  status: {
    type: String,
    enum: ['pending', 'paid', 'shipped', 'cancelled'],
    default: 'pending'
  }
}, { timestamps: true });

// Index composé pour optimiser les recherches fréquentes par client et statut
OrderSchema.index({ customer: 1, status: 1 });

export const OrderModel = model<IOrder>('Order', OrderSchema);

```

4.2 Base Relationnelle PostgreSQL avec Prisma ORM

Prisma garantit un typage de bout en bout (Type-safety) synchronisé directement avec le schéma de base de données SQL.

```
// schema.prisma
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum Role {
  USER
  ADMIN
}

model User {
  id          String   @id @default(uuid())
  email       String   @unique
  name        String?
  password    String
  role        Role     @default(USER)
  posts       Post[]
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Post {
  id          String   @id @default(uuid())
  title       String
  content     String?
  published   Boolean  @default(false)
  authorId    String
  author      User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
}
```

MODULE 5 : DevOps, Containerisation & CI/CD

5.1 Multi-Container orchestration avec Docker Compose

Docker permet d'isoler l'application et ses dépendances dans des conteneurs identiques entre le développement et la production.

```
# docker-compose.yml
version: '3.8'

services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "5000:5000"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgresql://postgres:secret@postgres:5432/app_db?schema=public
      - REDIS_URL=redis://redis:6379
    depends_on:
      - postgres
      - redis
    restart: always

  postgres:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: app_db
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

MODULE 6 : Projets Pratiques Complètement Réalisés

Projet 1 : TaskManager Pro (API REST + React + Auth JWT)

Objectif : Concevoir un système complet de gestion de projets d'équipe.

Spécifications Techniques :

- Authentification JWT avec Refresh Token stocké dans un cookie HTTPOnly.
- CRUD complet pour l'organisation de tâches sous forme de tableau Kanban.
- Filtres dynamique en temps réel sur React.

🎯 Projet 2 : E-Commerce Platform Lite (Prisma + Stripe + S3)

Objectif : Créer une boutique en ligne sécurisée acceptant les paiements par carte.

- Intégration du SDK Stripe Elements côté client.
- Gestion des *Webhooks Stripe* backend pour traiter les événements de paiement asynchrones.
- Stockage des images produits sur Amazon S3 / Cloudinary via Multer.

🎯 Projet 3 : Real-Time Chat Engine (Socket.io + Redis)

Objectif : Développer un serveur de messagerie instantanée à forte charge.

```
// Backend Socket.io avec Express
import { Server } from 'socket.io';
import http from 'http';
import express from 'express';

const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: { origin: "*" }
});

io.on('connection', (socket) => {
  console.log(`Nouvel utilisateur connecté : ${socket.id}`);

  socket.on('join_room', (roomId) => {
    socket.join(roomId);
    console.log(`User ${socket.id} joined room ${roomId}`);
  });

  socket.on('send_message', (data) => {
    // Broadcast à tous les membres du salon sauf l'émetteur
    socket.to(data.roomId).emit('receive_message', {
      message: data.message,
      sender: data.sender,
      timestamp: new Date()
    });
  });

  socket.on('disconnect', () => {
    console.log(`Utilisateur déconnecté : ${socket.id}`);
  });
});

server.listen(4000, () => console.log("Chat Server Running on port 4000"));
```